

Rust - Cargo

Présentation

Cargo s'occupe de compiler le code, de télécharger les bibliothèques dont le code dépend, et de compiler ces bibliothèques.

Cargo s'installe avec l'[installateur officiel](#) de [Rust](#). Vérifier l'installation avec `cargo --version`.

Créer un projet avec **Cargo**

```
cargo new my_project
cd my_project
```

`cargo new nom_projet` crée un projet avec des fichiers insérés dans le répertoire prenant le nom du projet :

- un fichier `Cargo.toml`
- un répertoire `src` qui contient le fichier `main.rs`
- un dépôt [Git](#) + le fichier `.gitignore` (l'option `--vcs=None` empêche la création d'un dépôt **Git**)

Contenu de `Cargo.toml` :

```
[
  package
]
name = "my_project"
version = "0.1.0"
edition = "2024"

[
  dependencies
]
```

Sous l'en-tête `[package]` nous trouvons les informations de configuration dont **Cargo** a besoin pour compiler le programme. Sous `[dependencies]` l'éventuelle liste des dépendances du projet.

Le fichier `main.rs` contient les lignes suivantes, automatiquement générées :

```
fn main() {
    println!("Hello, world!");
}
```

Compilation d'un projet **Cargo**

```
cargo build
```

Cette commande crée un fichier exécutable dans `target/debug/my_project` que nous pouvons lancer avec `./target/debug/my_project`.

Lancer `cargo build`, la première fois, amène **Cargo** à créer le fichier `Cargo.lock` à la racine du répertoire du projet, fichier qui garde une trace des versions exactes des dépendances du projet.

```
cargo run
```

Cette commande permet à la fois de compiler le code et ensuite de lancer l'exécutable.

```
cargo check
```

Cette commande vérifie rapidement que le code est bien compilable, mais ne produit pas d'exécutable, ce qui permet de vérifier plus rapidement qu'avec une compilation (pratique pour une vérification continue de son travail).

Commandes Cargo

- Vérifier installation : `cargo --version`
- Création projet : `cargo new nom_projet` (voir également `cargo new --help` pour les options possibles)
- Création sans dépôt [Git](#) : `cargo new --vcs=None nom_projet`
- Création d'une bibliothèque : `cargo new --lib ma_bibliotheque`
- Aide : `cargo new --help` (ou autre commande)
- Vérifier si projet compilable : `cargo check`
- Compilation de développement : `cargo build`
- Exécution : `./target/debug/nom_projet`
- Compilation et exécution : `cargo run`
- Compilation pour distribution : `cargo build --release`
- Exécution : `./target/release/nom_projet`
- Récupérer projet sur dépôt [Git](#) :

```
$ git clone https://depot.org/le_projet
$ cd le_projet
$ cargo build
```

- Met à jour les [crates](#) dans les limites définies des versions : `cargo update`
- Accès (via le navigateur) à la documentation des dépendances du projet : `cargo doc --open`
- Lancer des tests (avoir un répertoire `tests` avec fichiers de tests à la racine du projet) : `cargo test`